

AI Music Recommender — Complete Code Reference

For the Neo Technical Deep Dive with Igor

How to use this doc: This is your study sheet. For every constant, function, and test, you get: what it is, the actual code lines, a plain explanation, and **how to talk about it**. At the end of each section there's a "**Show Igor**" note telling you whether that piece belongs on the whiteboard, a slide, or the raw code.

The one mental model that organizes everything:

`recommender.py` is the **deterministic engine** — pure math, no AI. `agent.py` is the **intelligence layer** — all four Gemini calls live here. The two JSON files are the **RAG knowledge base**. Every question Igor asks maps to one of these three.

PART 1 — `agent.py` (the intelligence layer, 332 lines)

Everything the slides call *parsing*, *checking*, *adjusting*, and *generating* lives in this file. Nine functions, walked in the order the loop runs them.

1.1 — The constants (lines 26–43)

```
GEMINI_MODEL = "gemini-2.5-flash"           # line 26
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY") # line 27

DEFAULTS = {                                # line 30
    "favorite_genre": "", "favorite_mood": "",
    "target_energy": 0.5, "target_valence": 0.5,
    "likes_acoustic": False, "target_instrumentalness": 0.1,
}

MIN_SCORE_THRESHOLD = 3.0                    # line 39

HARMFUL_KEYWORDS = [                        # line 41
    "kill", "murder", "suicide", "hate", "abuse", "rape", "bomb", "terrorist"
]
```

What each group does:

- **GEMINI_MODEL** — the exact model. If Igor asks "which model?" the answer is *gemini-2.5-flash*, chosen for speed and a free tier.
- **DEFAULTS** — the safety net for parsing. If Gemini leaves a field blank, these values fill in so the scorer never hits a missing key. Note the neutral 0.5s and low 0.1 instrumentalness (most people want vocals).
- **MIN_SCORE_THRESHOLD = 3.0** — the single number behind the **score guardrail**. Below this, the system warns instead of pretending.
- **HARMFUL_KEYWORDS** — the 8-word blocklist for the **input guardrail**.

How to talk about it: "I keep the tunable values as named constants at the top — the model name, the score threshold, the safety defaults, and the harmful-word list — so behavior is easy to reason about and change in one place."

Show Igor: Nothing visual. Mention these verbally only if he asks about the threshold or model.

1.2 — `_validate_input` (line 46) — the INPUT GUARDRAIL

```
def _validate_input(user_input: str) -> tuple[bool, str]:    # line 46
    if not user_input or not user_input.strip():            # empty
        return False, "Input is empty."
    words = user_input.strip().split()
    if len(words) < 3:                                       # too short
        return False, "...at least 3 words."
    real_words = [w for w in words if any(c.isalpha() for c in w)]
    if len(real_words) < 2:                                   # nonsense (line 59-
60)
        return False, "...describe the music you want in plain words."
    lower = user_input.lower()
    for keyword in HARMFUL_KEYWORDS:                         # harmful
        if keyword in lower:
            return False, "...keep your request music-related."
    return True, ""
```

What it does: Four checks, in order — empty → under 3 words → fewer than 2 words containing actual letters (the "123 !!! 456" nonsense case) → contains a harmful keyword. Returns a `(is_valid, reason)` pair. **Pure Python, zero AI.**

Why it matters: This is what "reject before Gemini" literally means. It runs *first* in `run_agent`, so junk never costs an API call.

How to talk about it: "The first guardrail is deterministic and runs before any API call — it catches empty input, one-word requests, gibberish with no real words, and a harmful-keyword blocklist. Cheap, instant, and it protects the paid API path."

Show Igor: Slide 12 (the four-guardrail slide). If he wants detail, point at this function in the **code** — it's short and self-explanatory.

1.3 — `_call_gemini` (line 72) — the shared API wrapper

```
def _call_gemini(prompt: str) -> str:                        # line 72
    response = client.models.generate_content(model=GEMINI_MODEL, contents=prompt)
    return response.text.strip()
```

What it does: The single choke-point for every Gemini call. All "3–5 API calls per query" flow through this two-line function.

How to talk about it: "Every LLM call goes through one wrapper. So when I say 'Gemini parses' or 'Gemini checks,' each of those is a prompt built elsewhere and handed to this one function. It also makes the system easy to test — I can patch this single point to simulate a failure." (*That patching is literally how two of your integration tests work.*)

Show Igor: The **code**, but only if he asks "how do you actually call the model?" Otherwise skip.

1.4 — `_parse_profile` (line 77) — this is PARSING

```
def _parse_profile(user_input: str) -> Dict: # line 77
    prompt = f"""... Return ONLY a valid JSON object with these exact keys:
    - favorite_genre ... - favorite_mood (one of: happy, chill, intense,
      melancholic, energetic, peaceful, dark ...)
    - target_energy (float 0.0-1.0) ... - target_valence ...
    - likes_acoustic (boolean) - target_instrumentalness ...
    - confidence ("high" ... or "low" ...)
    User request: "{user_input}" ...""" # lines 79-93
    raw = _call_gemini(prompt)
    raw = raw.replace("```json", "").replace("```", "").strip() # line 98
    try:
        parsed = json.loads(raw) # line 101
        return parsed
    except json.JSONDecodeError: # line 104
        return {} # safe empty on failure
```

What it does: Turns free text ("something chill to study to") into a structured dict with fixed keys. The prompt *constrains mood to 7 allowed labels* — the same 7 in your mood knowledge base. Line 98 strips the markdown fences models love to add. Line 104 is the **output guardrail in miniature**: bad JSON returns `{}` instead of crashing.

The confidence field is important — it's what drives the follow-up question later. "low" confidence → the agent asks instead of guessing.

How to talk about it: "Parsing is the first Gemini call. I give it a strict schema — seven keys, mood constrained to a fixed vocabulary — and ask for JSON only. I strip code fences and defensively catch bad JSON, returning an empty dict that the defaults then backfill. The **confidence** flag it returns is what later decides whether to ask a follow-up."

Show Igor: Slide 4, step 2 ("Gemini parses request"). If he probes, open the **code** and show the prompt schema (lines 80–88) — the constrained key list is impressive.

1.5 — `_check_quality` (line 109) — this is CHECKING ★ YOUR CENTERPIECE

```
def _check_quality(user_input: str, results: List[Tuple]) -> str: # line 109
    songs_summary = "\n".join(
```

```

        f"- {song['title']} by {song['artist']} | genre: {song['genre']} "
        f"| mood: {song['mood']} | energy: {song['energy']:.2f}"
    for song, _, _ in results[:3]                # top 3 only (line 113)
)
prompt = f"""A user asked for music with this request: "{user_input}"
The recommender returned these top songs:
{songs_summary}
Do these results match what the user asked for?
Reply with ONLY one word: "good" or "retry"."""    # lines 115-122
answer = _call_gemini(prompt).lower()
return "good" if "good" in answer else "retry"      # line 126

```

What it does: After scoring, *before* answering, it hands Gemini the **original request** plus the **top 3 results** and asks a single yes/no question. Line 126 is the decision: only a clear "good" passes; **anything else becomes a retry** (fail-safe toward retrying).

Why this is the heart of the project: The scorer is deterministic — it will always rank *something* highest, even from a bad batch. This function is a *second, independent judgment* that asks "do these actually match the intent?" It's the thing that catches the LLM's tendency to hallucinate agreement. **The generator is not the approver.**

The honest nuance to volunteer: it uses the same model that generates, so it's not fully independent — but it's a *different task* (evaluating, not generating) seeing only the request and results. It reliably catches obvious mismatches; a stronger version would use a different model or a scored rubric.

How to talk about it: (*This is your 3 minutes — go slow.*) "Here's the part I care about most. After scoring, I don't just return the top results — I send the original request and the top 3 back to Gemini and ask one question: do these match? One word back, good or retry. The reason this is separate from the scorer is that the scorer is just math — it always ranks something first even when the whole batch is weak. This is a second opinion that judges intent. It's what catches the model rubber-stamping bad results. And I fail toward retry — if the answer is anything but a clear 'good,' I loosen constraints and try again."

Show Igor: This is the **whiteboard moment**. Draw the loop (see Part 4). The check is the **diamond** that either exits or loops back. Slide 5 is your backup if you fumble. If he wants proof, the **code** — line 126 is the whole decision in one line.

1.6 — `_relax_profile` (line 129) — this is ADJUSTING

```

def _relax_profile(profile: Dict, attempt: int) -> Dict:    # line 129
    relaxed = profile.copy()
    if attempt == 1:                                       # 1st retry
        current = relaxed.get("target_energy", 0.5)
        relaxed["target_energy"] = current * 0.7 + 0.5 * 0.3    # pull toward 0.5
    elif attempt == 2:                                     # 2nd retry
        relaxed["favorite_genre"] = ""                        # drop genre
    return relaxed

```

What it does: Defines *how* the loop loosens up. First retry: nudge energy 30% of the way toward neutral 0.5 (so an "extreme" request stops over-constraining). Second retry: drop the genre requirement entirely, opening the whole catalog.

The off-by-one to know cold: attempt index 0 runs *before* any relaxing, so relaxing kicks in at attempt 1 and 2. That's why "3 retries" really means "1 original try + 2 relaxations."

How to talk about it: "When the check says retry, I don't just re-run the same thing — I relax a constraint. First I pull the energy target toward the middle, because extreme requests over-filter. If that still fails, I drop the genre requirement to widen the pool. It's a deliberate escalation from least to most permissive."

Show Igor: Slide 10 (Case 2 shows RETRY 1 → widen energy, RETRY 2 → drop genre — those labels come straight from this function). On the whiteboard, this is the arrow looping back from the check to the score step.

1.7 — `_load_context_db` + the two loads (lines 144–155) — RETRIEVAL setup

```
def _load_context_db(path: str) -> Dict:                                # line 144
    try:
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    except Exception:
        return {}

_GENRE_CONTEXT = _load_context_db("data/genre_context.json")          # line 154
_MOOD_CONTEXT  = _load_context_db("data/mood_context.json")           # line 155
```

What it does: Loads the two knowledge bases into memory **once**, at import time (not per request — that's an efficiency choice). Fails safe to `{}` so a missing file degrades gracefully instead of crashing.

How to talk about it: "The RAG knowledge bases load once at startup, not per query. They're the 'retrieval' source — genre and mood descriptions I can pull by key. If they're missing, the system still runs; explanations just get less rich."

Show Igor: Slide 8 (the RAG slide lists these two JSON sources). Show the **actual JSON** if he wants to see what "retrieval" returns (see Part 3).

1.8 — `generate_rag_explanations` (line 158) — this is the RAG / GENERATING step

```
def generate_rag_explanations(user_input, results):                    # line 158
    songs_context = ""
    for i, (song, score, fallback_explanation) in enumerate(results, start=1):
        genre_desc = _GENRE_CONTEXT.get(song["genre"], "")            # retrieve (169)
        mood_desc  = _MOOD_CONTEXT.get(song["mood"], "")              # retrieve (170)
```

```

songs_context += f"""\Song {i}: {song['title']} ...
Energy: {song['energy']:.2f} | Valence: {song['valence']:.2f} ...""" #
172-177
if genre_desc: songs_context += f"\n Genre context: {genre_desc}"
if mood_desc: songs_context += f"\n Mood context: {mood_desc}"
prompt = f"""\... For each song, write a single natural sentence ...
Be specific – mention actual feature values ...
Return ONLY a JSON array of {len(results)} strings ...""" # 185-195
try:
    raw = _call_gemini(prompt) # ONE call, all songs
(199)
    raw = raw.replace("`json", "").replace("`", "").strip()
    explanations = json.loads(raw)
    if isinstance(explanations, list) and len(explanations) == len(results):
        return [(song, score, explanations[i]) # 204-207
                for i, (song, score, _) in enumerate(results)]
except Exception as e:
    logger.warning(f"RAG explanation failed, using fallback: {e}")
return results # fallback (211)

```

What it does, in two halves:

1. **Retrieval (the for-loop, 168–183):** for each recommended song it gathers three things — the song's *own* audio features (energy, valence, acousticness, etc.), the *genre* description from JSON, and the *mood* description from JSON. That bundle is the "grounding."
2. **Generation (185–207):** one **batched** Gemini call for *all* songs, asking for a JSON array of one-sentence explanations that cite real values. Line 202 validates the array length matches before using it.

The fallback (line 211) is a guardrail: if Gemini fails or returns the wrong number of items, it returns the original results with their score-based explanation strings — so the user always gets *an* explanation, never a crash.

Why "one call for all 5" is worth saying: it's why a clear request is only 3 API calls total, not 7. Batching keeps latency and cost down.

How to talk about it: "The RAG layer grounds every explanation. For each pick I retrieve three things — the song's real audio features and the genre and mood descriptions from my knowledge base — and send them all in *one* batched call asking for a sentence per song that cites actual numbers. If that call fails or comes back malformed, I fall back to the deterministic score explanation, so the user always gets something coherent."

Show Igor: Slide 8 for the concept, **Slide 9** (Case 1) for the real grounded output citing 0.89 energy. Show the **code** fallback line (211) if he asks "what if the LLM fails?"

1.9 — `run_agent` (line 219) — THE ORCHESTRATOR (the whole loop)

This is the function you draw on the whiteboard. The path in order:

```

def run_agent(user_input, songs, k=5): # line 219
    is_valid, reason = _validate_input(user_input) # 230 GUARDRAIL

```

```

    if not is_valid: return None
    profile = _parse_profile(user_input) # 240 PARSE (call 1)
    if not profile: return None
    # follow-up ONLY if BOTH genre and mood are empty:
    if not profile.get("favorite_genre") and not profile.get("favorite_mood"): #
253
        followup = input("Could you tell me the genre or mood...") # 255
        extra = _parse_profile(followup) # 257 PARSE (call 2,
case 3)
        ... merge genre/mood ...
        for key, default in DEFAULTS.items(): ... # 267 backfill
defaults
        best_results = None
        for attempt in range(3): # 273 HARD CAP = 3
            results = recommend_songs(profile, songs, k=k, mode="default") # 275
SCORE
            ...
            quality = _check_quality(user_input, results) # 288 CHECK (1
call/attempt)
            if quality == "good": # 289
                final = generate_rag_explanations(...) # 293 EXPLAIN (1 call)
                _score_guardrail(final) # 294
                return final
            profile = _relax_profile(profile, attempt) # 311 ADJUST, loop
back
        # after 3 failed attempts – honest fallback:
        print("I couldn't find a perfect match, but here's the closest I found:") #
316
        final = generate_rag_explanations(user_input, best_results) # 317
        _score_guardrail(final)
        return final

```

What it does: Orchestrates plan → act → check → adjust. Key decisions to know:

- **Follow-up (253):** triggers *only* when both genre AND mood came back empty — that's the low-confidence path (Case 3). It costs a **second parse call**.
- **for attempt in range(3) (273):** the hard cap. The loop can never run forever — that's the **execution guardrail**.
- **On "good" (289–296):** explain, run score guardrail, return.
- **After 3 fails (313–320):** the honest fallback path — prints "I couldn't find a perfect match" and returns the best it found (Case 2's ending).

This is where the API-call math comes from:

- Clear request = parse(1) + check(1) + explain(1) = **3 calls**
- Follow-up = parse(1) + parse(1) + check(1) + explain(1) = **4 calls**
- Full 3-retry = parse(1) + check×3 + explain(1) = **5 calls**

How to talk about it: "run_agent" ties it together. Guardrail, parse, an optional follow-up if confidence is low, then the retry loop — score, check, and either return or relax and try again, capped at three. If all three

attempts fail, it returns the best batch with an honest 'couldn't find a perfect match' message rather than faking confidence."

Show Igor: WHITEBOARD. This whole function is the diagram you draw. Slide 4 backs it up. Walk your finger down the numbered path as you drew it.

1.10 — `_score_guardrail` (line 323) — the SCORE GUARDRAIL

```
def _score_guardrail(results):                                # line 323
    if not results: return
    top_score = results[0][1]                                # top score
    if top_score < MIN_SCORE_THRESHOLD:                       # < 3.0
        print(f"⚠ Warning: The best match found scored {top_score:.2f} / 6.5, "
              f"which is below our quality threshold ...")    # 329-331
```

What it does: After results are final, checks the top score against 3.0. If below, prints the honest warning. Pure Python, no AI.

How to talk about it: "The last guardrail is about honesty. If even the best match scores under 3 out of 6.5, the system says so out loud instead of presenting a weak result as if it were great. Naming what it can't do is part of what it does."

Show Igor: Slide 10 (Case 2 — the gold warning banner is this function firing). Slide 12 lists it as the 4th guardrail.

PART 2 — `recommender.py` (the deterministic engine, 309 lines)

No AI anywhere in this file. This is the "dumb, fully testable" half. Same input always gives the same output — which is *why* it's testable.

2.1 — `Song` / `UserProfile` dataclasses + `Recommender` class (lines 4–101)

What they are: `Song` (line 4) and `UserProfile` (line 22) are simple typed containers. `Recommender` (line 35) is an OOP wrapper with `recommend()` and `explain_recommendation()` methods.

The honest point to know: all three docstrings literally say "Required by tests/test_recommender.py." The **live pipeline doesn't use this class** — `run_agent` calls the module-level functions (`recommend_songs`, `score_song`) directly on plain dicts. The class exists to give the unit tests a clean object-oriented surface.

How to talk about it (if asked "why both a class and functions?"): "The live path uses lightweight dicts and module-level functions for speed over 114k rows. The `Recommender` class is a thin OOP wrapper I kept for the unit tests — it makes them readable. So there are two entry points to the same scoring logic: the class for tests, the functions for production."

Show Igor: Only the **code**, and only if he asks about the class/function duplication. Don't volunteer it.

2.2 — `load_songs` (line 103) — DEAD CODE (the two-datasets answer)

What it is: The *old* loader for the 29-song `songs.csv`.

The critical fact: nothing calls it. The live system uses `load_songs_v2`. This is the "why are there two datasets?" trap.

How to talk about it (rehearse this until it's smooth): "The 29-song set and this old loader are legacy from the Module 3 starter. The live system runs entirely on the 114k Spotify set through `load_songs_v2` — this old function isn't called anywhere. I kept the file only for provenance; it's effectively dead code I'd delete in a cleanup."

Show Igor: Nothing — this is a *verbal* answer you give if asked. Do **not** put it on a slide. Just know it cold.

2.3 — `derive_mood` (line 126) — THE CIRCUMPLEX MODEL

```
def derive_mood(energy, valence, mode, tempo, acousticness): # line 126
    if valence >= 0.6 and energy >= 0.6: mood = "happy" # 129
    elif valence >= 0.6 and energy < 0.4: mood = "peaceful" # 131
    elif valence < 0.4 and energy >= 0.6: # 133
        mood = "dark" if mode == 0 else "intense" # minor key → dark
    elif valence < 0.4 and energy < 0.4: mood = "melancholic" # 136
    elif valence >= 0.5 and energy >= 0.4: # 138
        mood = "energetic" if tempo >= 120 else "chill" # tempo refines
    else:
        mood = "chill" if acousticness >= 0.5 else "intense" # 143
    return mood
```

What it does: The Spotify dataset has **no mood column** — only valence and energy as numbers. This function *creates* mood using the **circumplex model of affect** (Russell, 1980): valence (positive↔negative) on one axis, energy/arousal (calm↔excited) on the other. Four quadrants give the base mood, then three refinements: **mode** (major/minor key) splits dark vs intense, **tempo** splits energetic vs chill, **acousticness** breaks the last tie.

Why it's a great story: it's real feature engineering, and it's where a *psychology model* ends up doing your data work. Own the limitation: ~70–80% accurate, western-music bias — both in your model card.

How to talk about it: "The dataset gave me valence and energy as numbers but no mood label, and users think in mood words. So I derive mood with the circumplex model from affective psychology — the two axes are valence and energy, which map to four quadrants, and I refine the edges with key, tempo, and acousticness. It's about 70–80% accurate on spot-checks with a known western bias, which I document."

Show Igor: Slide 7 (the quadrant diagram). If he's curious, the **code** — the branch structure reads cleanly. The `KNOWN_MOODS` test (2.9 below) proves the accuracy claim.

2.4 — `load_songs_v2` (line 148) — THE LIVE LOADER

```
def load_songs_v2(csv_path, sample=None): # line 148
    for i, row in enumerate(reader):
        try:
            energy = float(row["energy"]); valence = float(row["valence"]); ...
            for col, val in [...]: # range check 168-172
                if not (0.0 <= val <= 1.0): raise ValueError(...)
        except (ValueError, KeyError):
            continue # DROP bad rows (174-175)

    songs.append({
        "title": row["track_name"], "artist": row["artists"], # column
        "genre": row["track_genre"], ... # 179-181
        "mood": derive_mood(energy, valence, mode, tempo, acousticness), #
    }) # 188

    if sample is not None:
        songs = random.sample(songs, min(sample, len(songs))) # 191-192
    return songs
```

What it does: Reads the 114k Spotify CSV and does three jobs: (1) **maps Spotify's column names** to yours (`track_name`→`title`, `artists`→`artist`, `track_genre`→`genre`, lines 179–181); (2) **cleans data** — the try/except drops any row with missing fields or values outside 0–1 (168–175); (3) **derives mood** for every surviving row (188). The optional `sample` (191) is how tests run on 2k/5k rows instead of all 114k.

How to talk about it: "`load_songs_v2` is the real loader. It maps Spotify's schema to mine, drops rows with missing or out-of-range values so the scorer never sees dirty data, and derives a mood for every track as it loads. The optional sample parameter is what lets my tests run on a few thousand rows quickly."

Show Igor: The **code** if he asks about data cleaning or the 114k. Otherwise it's covered implicitly by Slide 2 ("114,000 real tracks").

2.5 — `SCORING_MODES` (line 197) — the weight table

```
SCORING_MODES = { # line 197
    "default": {
        "genre": 1.0, "mood": 1.0, "energy": 2.0, "valence": 0.5, "acousticness": 0.5, "instrumentalness": 0.5},
    "genre-first": {"genre": 3.0, ...},
    "mood-first": {"mood": 3.0, ...},
    "energy-first": {"energy": 4.0, ...},
}
```

What it does: Four named weight presets. In **default** (the only one the live pipeline uses — agent.py line 275 passes `mode="default"`), **energy is weighted 2.0**, the highest, so energy-closeness dominates the score. The other three presets exist to show the design flexes.

How to talk about it: "Scoring weights live in a table so they're tunable. The default weights energy highest, since it's the dimension people care most about for a vibe. There are alternate presets — genre-first, mood-first — that re-weight the same formula; the live path uses default."

Show Igor: Slide 6 mentions the modes in the footer. Show the **code** table only if he asks "how are things weighted?"

2.6 — `score_song` (line 233) — THE SCORING FORMULA

```
def score_song(user_prefs, song, mode="default"):          # line 233
    weights = SCORING_MODES.get(mode, SCORING_MODES["default"])
    score = 0.0
    if song["genre"] == user_prefs["favorite_genre"]:      # exact-match bonus
        score += weights["genre"]                         # 240-242
    if song["mood"] == user_prefs["favorite_mood"]:         # exact-match bonus
        score += weights["mood"]                           # 245-247
    energy_score = weights["energy"] * (1 - abs(song["energy"] -
user_prefs["target_energy"]))
    score += energy_score                                  # closeness (250)
    valence_score = weights["valence"] * (1 - abs(song["valence"] -
user_prefs["target_valence"]))
    score += valence_score                                # closeness (255)
    if user_prefs["likes_acoustic"]:                       # DIRECTIONAL (260-
264)
        acoustic_score = weights["acousticness"] * song["acousticness"]
    else:
        acoustic_score = weights["acousticness"] * (1 - song["acousticness"])
    score += acoustic_score
    instr_score = weights["instrumentalness"] * (1 - abs(song["instrumentalness"]
- user_prefs["target_instrumentalness"]))
    score += instr_score                                  # closeness (268)
    return score, " | ".join(reasons)
```

What it does: Scores one song against the profile. Three kinds of terms:

- **Exact-match bonuses** — genre and mood: full weight if equal, else nothing.
- **Closeness terms** — energy, valence, instrumentalness: $\text{weight} \times (1 - |\text{target} - \text{actual}|)$. This is the "1 minus the difference" from your slide — identical values give the full weight, far-apart values give near zero.
- **Directional term** — acousticness: rewards high acousticness if they like acoustic, rewards *low* if they don't (260–264).

Returns `(score, explanation_string)`. Max possible ≈ 6.5 (sum of all weights when everything matches perfectly) — that's where `"/ 6.5"` comes from.

How to talk about it: "Each song gets a weighted score. Genre and mood are exact-match bonuses. Energy, valence, and instrumentality use a closeness formula — weight times one minus the absolute difference — so nearer is better. Acousticness is directional depending on preference. Sum it up, max is about 6.5."

Show Igor: Slide 6 (the scoring-engine slide). The closeness formula is right there. Show the **code** if he wants to see the directional acousticness branch.

2.7 — `recommend_songs` (line 275) — SCORE ALL → SORT → DIVERSIFY

```
def recommend_songs(user_prefs, songs, k=5, mode="default"): # line 275
    scored = [(song, *score_song(user_prefs, song, mode)) for song in songs] #
    278-280
    ranked = sorted(scored, key=lambda x: x[1], reverse=True) # 282 sort desc
    results, artist_counts, genre_counts, seen_titles = [], {}, {}, set()
    for song, score, explanation in ranked:
        if (artist, title) in seen_titles: continue # dedupe (294)
        if artist_counts.get(artist, 0) >= 2: continue # max 2/artist (296)
        if genre_counts.get(genre, 0) >= 2: continue # max 2/genre (298)
        results.append((song, score, explanation))
        ... update counts ...
        if len(results) == k: break # stop at k (306)
    return results
```

What it does: Scores every song, sorts by score descending, then walks down the ranked list applying the **diversity cap**: skip if this (artist,title) was already seen, skip if that artist already appears twice, skip if that genre already appears twice. Stops once it has k. **This is literally where "max 2 per artist, 2 per genre" happens.**

Why the cap matters: without it, a perfect-matching artist could fill all 5 slots. The cap forces variety.

How to talk about it: "`recommend_songs` scores all 114k, sorts, then applies a diversity cap as it fills the top k — no more than two songs per artist or per genre, and no duplicate titles. Without that, one artist who matches perfectly would take every slot. It trades a little raw score for a more useful, varied list."

Show Igor: Slide 6 (footer mentions the cap). The **code** loop (289–307) is clean if he wants to see the cap logic.

PART 3 — The JSON knowledge bases (the RAG source)

3.1 — `genre_context.json`

- A **dict of 38 genres** → plain-English description.
- Example: "`pop`" → *"Pop music is built around catchy melodies, polished production, and broad mainstream appeal..."*

- **Note:** your slide says "~40" — it's **38**. Say "about 40" or just "38."

3.2 — mood_context.json

- A **dict of 7 moods** → description written *explicitly in circumplex terms*.
- Example: "**happy**" → "...occupies the high-valence, high-arousal quadrant of the circumplex model of affect..."
- **Why this matters:** these 7 moods are the *same* 7 that **derive_mood** produces and that **_parse_profile** constrains Gemini to. The knowledge base, the derivation, and the parser all speak the same mood vocabulary, grounded in the same model. That consistency is a design strength worth pointing out.

How to talk about it: "The retrieval source is two JSON files — 38 genre descriptions and 7 mood descriptions. The moods are written in circumplex terms, the same model my derivation uses, so the whole system shares one vocabulary: the parser constrains to these 7 moods, the derivation produces them, and the RAG layer explains with them."

Show Igor: Slide 8 names both files. If he asks "what does retrieval actually pull?", open the **actual JSON** — one genre entry and one mood entry make it concrete instantly.

PART 4 — The three test files (21 tests)

The honest headline number: 21 tests — 15 reliability, 4 integration, 2 unit. 17 run with no API key in ~12 seconds. (Not 22 — the old slide had a typo.)

4.1 — test_recommender.py (2 UNIT tests)

Builds a tiny 2-song **Recommender** by hand (a pop/happy track and a lofi/chill track), no dataset, no API.

- **test_recommend_returns_songs_sorted_by_score (line 35)** — asks for a pop/happy/high-energy profile, asserts 2 results come back and the top one is the pop+happy track. **Proves ranking works.**
- **test_explain_recommendation_returns_non_empty_string (line 51)** — asserts the explanation is a non-empty string. **Proves explanations generate.**

How to talk about it: "Two pure unit tests on a hand-built two-song recommender — one checks that scoring orders correctly, one checks that explanations aren't empty. No data, no API, instant."

4.2 — test_reliability.py (15 tests, no API) — the meat

Loads a fixed 5,000-row sample with **SEED = 42** so results are reproducible.

- **test_consistency (26)** — runs the same profile 5 times, asserts identical top-5 every run. **Proves determinism.** (This is your "same input, same output" claim, tested.)
- **test_score_threshold_good_match (51)** — a clean pop/happy profile must score ≥ 3.0 . Proves good requests clear the bar.
- **test_score_threshold_flags_poor_match (69)** — a deliberately conflicting profile; logs whether it falls below threshold. Documents the weak-match case.

- **test_mood_derivation_accuracy (103)** — runs `derive_mood` against **7 hand-labeled songs** (`KNOWN_MOODS`, line 92) and asserts $\geq$ **70% accuracy**. **This is what backs your "70–80%" claim — it's measured, not guessed.**
- **Four edge cases (121–176):** empty genre, unknown genre ("jazz fusion"), all-midpoint 0.5 preferences, and a rare genre ("tango"). Each asserts the system returns results / doesn't crash. **Proves robustness to weird input.**
- **test_precision_mood_match (182)** — asserts $\geq$ **40%** of the top 5 match the intended mood or genre. **This is your precision metric.**
- **Five guardrail tests (209–250):** empty, too-short, nonsensical, harmful — all must be *rejected*; and one valid input must be *accepted*; plus a score-threshold guardrail check. **Proves each input guardrail branch works.**

How to talk about it: "The reliability suite is the bulk — 15 tests, no API. The important ones: a consistency test that proves determinism, a mood-accuracy test against seven hand-labeled songs that backs my 70–80% number, a precision test requiring at least 40% relevant in the top 5, four edge cases, and five guardrail tests covering every rejection branch."

4.3 — `test_integration.py` (4 tests, live Gemini)

Uses a 2,000-row sample. These make real API calls.

- **test_agent_retries_on_poor_match (26)** — feeds a contradictory request ("extremely high energy dark melancholic jazz fusion"), asserts the agent still returns results without crashing. **Proves the retry loop survives conflict.**
- **test_rag_fallback_on_gemini_failure (39)** — **patches `_call_gemini` to raise an exception** (line 53), then asserts `generate_rag_explanations` still returns the right number of non-empty explanations. **Proves the RAG fallback works** — and shows why the single `_call_gemini` wrapper is handy for testing.
- **test_end_to_end_full_flow (66)** — a normal request through the whole pipeline; asserts every result is a `(dict, float, str)` with a real explanation and positive score. **Proves the output shape is well-formed end to end.**
- **test_guardrail_blocks_before_gemini (86)** — **mocks Gemini**, sends harmful input, asserts the result is `None` and `mock_gemini.assert_not_called()` (line 94). **Proves harmful input never reaches the API** — the guardrail truly runs first.

How to talk about it: "Four integration tests hit the live model. Two use mocking cleverly — one patches the Gemini call to throw, proving the RAG fallback; another mocks Gemini entirely and asserts it's *never called* on harmful input, proving the guardrail runs before any API cost. Plus a conflict-retry test and a full end-to-end shape check."

4.4 — The 3 real bugs (say this if asked "did tests catch anything?")

"Three real bugs before shipping: a dict-vs-`Song` type mismatch that crashed the pipeline, a duplicate track appearing under two genre tags, and mood boundary mislabels. The type mismatch is why I now test the seams between components, not just each piece."

Show Igor: Slides 12–13. If he digs into testing philosophy, the two **mocking tests** (integration 2 and 4) are the most impressive — show that **code**.

PART 5 — DECISION TABLE: whiteboard vs slide vs code

Topic	Whiteboard	Slide	Show code
The overall loop (plan/act/check/adjust)	✓ draw it live	4 (backup)	run_agent if pushed
The checker (<code>_check_quality</code>)	✓ the decision diamond	5	line 126
Retry / relax logic	✓ the loop-back arrow	10	<code>_relax_profile</code>
Input guardrail	—	12	<code>_validate_input</code>
Score guardrail	—	10, 12	<code>_score_guardrail</code>
Parsing	—	4	prompt schema 80–88
Scoring formula	—	6	<code>score_song</code>
Diversity cap	—	6	loop 289–307
Mood derivation	—	7	<code>derive_mood</code>
RAG grounding	—	8, 9	<code>generate_rag_explanations</code>
JSON knowledge base	—	8	the actual JSON
Two-datasets question	—	never slide	verbal answer only
Testing / mocking	—	12, 13	integration tests 2 & 4

The golden rule: structure and flow → **whiteboard**. Data, tables, and results → **slides**. Proof of a specific claim → **open the code** to that one function. Never read code line-by-line unprompted; go to it only when Igor wants proof.

PART 6 — Numbers to know cold

- Model: **gemini-2.5-flash**
- Tracks: **114,000** · genres in data: **125** · genre KB entries: **38** · mood labels: **7**
- API calls: **3** clear / **4** follow-up / **5** full-retry
- Score: max **6.5** · guardrail warns below **3.0**
- Retry cap: **3** attempts
- Tests: **21** total = **15** reliability + **4** integration + **2** unit · **17** run with no key in ~12s
- Mood accuracy: **~70–80%** (tested at ≥70%) · Precision: **≥40%** top-5 relevant
- Diversity cap: **2** per artist, **2** per genre

- 3 real bugs caught by tests